

# IMPACT ANALYSIS

## CR-016: Show / Hide Password Visibility Toggle in the Login Module

### 1. Change Summary

CR-016 introduces a client-side show/hide password visibility toggle to the Student Management System login module. Users and administrators will be able to toggle the password field between masked characters and plaintext view while entering their credentials. The change improves login usability and reduces authentication errors caused by typing mistakes, without altering backend security or authentication protocols.

### 2. Impact Assessment

| Area           | Impact         | Assessment                                                             |
|----------------|----------------|------------------------------------------------------------------------|
| Frontend       | Low - Moderate | Login form component state, toggle UI, and styling will be added.      |
| Backend        | None           | Existing API remains unchanged.                                        |
| API            | None           | Existing POST /api/auth/login endpoint remains in use.                 |
| Database       | None           | No schema or user credential changes required.                         |
| Authentication | Low            | Improves credential entry usability; backend authentication unaltered. |
| Authorization  | None           | Existing role-based protection and token storage remain unchanged.     |
| Performance    | Low            | Toggling is performed locally with minimal React state re-rendering.   |
| Usability      | High           | Eliminates typing mistakes and reduces login retry failures.           |
| Deployment     | Low            | No database migration or new backend endpoint required.                |

### 3. Frontend Impact

The main affected area is the login module located in the frontend client.

Potential changes include:

- Local visibility toggle state (`showPassword`) in `LoginFormSection.jsx`.
- Password input wrapper container for layout alignment.
- Interactive toggle button with SVG icons (eye / eye-slash).
- Right padding on password input to prevent text overlapping button.
- Accessible button labeling (`aria-label`, `aria-pressed`).
- Responsive styling adjustments in `login.css`.

The implementation must ensure activating the toggle button does not submit the form.

### 4. Backend, API and Database Impact

No backend changes are expected.

The existing:

POST /api/auth/login

endpoint will continue to receive credentials and return user details and tokens.

No new API endpoint or request payload modification is required.

No database schema, table, or user model changes are required.

## 5. Security Impact

The existing authentication and authorization mechanisms remain unchanged.

The password visibility toggle must:

- Default to masked state on load to mitigate shoulder-surfing risks.
- Not log or expose plaintext passwords in the browser console.
- Not store passwords in `localStorage`, `sessionStorage`, or cookies.
- Use standard React state handling within component memory.
- Preserve secure HTTPS transmission of credentials upon form submission.
- Maintain compatibility with browser autofill and credential managers.

## 6. Functional and Usability Impact

Existing login functionality for all user roles must continue to work seamlessly.

Particular attention is required for:

- Toggling visibility before, during, and after typing the password.
- Submitting the form while password is in the visible state.
- Submitting the form while password is in the masked state.
- Ensuring the toggle button uses `type="button"` to avoid premature submission.
- Preserving form validation when fields are submitted empty.
- Maintaining accurate error alert display on invalid credentials.

Filtering or toggling affects only visual presentation and does not alter credentials.

## 7. Performance Impact

The expected performance impact is negligible because state changes occur entirely in client-side memory.

No network requests or asynchronous calls are initiated when toggling password visibility.

Component re-rendering is scoped strictly to the password input field, ensuring zero impact on browser responsiveness or memory utilization.

## 8. Testing Impact

Manual testing should cover:

- Initial load masked state verification.
- Toggle to visible state (`type="text"`).

- Toggle back to masked state (`type="password"`).
- Icon state switching (eye to eye-slash and vice-versa).
- Password input legibility and spacing with long passwords.
- Form submission with valid credentials in masked mode.
- Form submission with valid credentials in visible mode.
- Form submission with invalid credentials (error handling).
- Form submission with empty username or password fields.
- Keyboard accessibility (Tab key navigation, Space/Enter activation).
- Screen reader accessibility (`aria-label`, `aria-pressed`).
- Responsive appearance on mobile, tablet, and desktop screens.
- Administrator login and navigation to `/dashboard`.
- Student login and navigation to `/student-dashboard`.

## 9. Risk Analysis

| Risk                                            | Impact | Mitigation                                                                               |
|-------------------------------------------------|--------|------------------------------------------------------------------------------------------|
| Shoulder-surfing / unauthorized viewing         | Medium | Default to masked mode; clear visual state indication.                                   |
| Toggle button prematurely triggers form submit  | Medium | Explicitly define <code>type="button"</code> on the toggle element.                      |
| Toggle icon covers password text                | Low    | Apply sufficient right padding to the password input field.                              |
| Inaccessible to keyboard or screen reader users | Low    | Implement <code>aria-label</code> , <code>aria-pressed</code> , and proper focus styles. |
| Login submission regression                     | High   | Perform regression testing for both Admin and Student authentication.                    |

## 10. Overall Impact and Recommendation

### Overall Impact: Low

CR-016 is primarily a frontend usability enhancement that improves user experience and minimizes mistyped credential errors during login. It does not require database, backend API, authentication mechanism, or authorization changes.

The change is technically feasible and strongly recommended for implementation, subject to formal approval and subsequent testing.

**Key implementation consideration:** Ensure the toggle button uses `type="button"` and input styling provides sufficient right padding so characters are never obscured behind the icon.